

KASPARRO

AI Representation Optimizer for Shopify

Technical Document — v1.0

Kasparro Agentic Commerce Hackathon | April 2026

Prepared by Aswin Kumar B S and Naveen D

Scope: full technical reference — architecture, schema, API, security, pipeline, AI stack, scheduler, failure handling, env vars

Stack: React 19 + Vite 7 | Express 5 + TypeScript | PostgreSQL (Neon) + Drizzle ORM

AI: Gemini 2.0 Flash (primary) -> OpenRouter Gemma 4 31B -> Groq LLaMA 3.3 70B | Vercel + Railway

16 database tables | 45+ REST endpoints | 5-platform GEO scanner | node-cron scheduler

1. System Architecture

1.1 Monorepo Package Map

Our project is a pnpm monorepo with four packages and two deployable artifacts. The architecture prioritises operational simplicity: persistent Node.js API (required for the scheduler and analysis queue) deployed on Railway, and a static SPA on Vercel. Complexity lives in the AI pipeline and the Shopify integration, not in the infrastructure layer.

Package / Artifact	Technology Stack	Responsibility
artifacts/ai-readiness	React 19, Vite 7, TailwindCSS v4, Radix UI, Recharts, React Query v5, Zustand	SPA frontend — dashboard, product list, fix review, GEO tracker, perception report, listing readiness, analysis trigger
artifacts/api-server	Express 5, TypeScript, Pino logger, node-cron, esbuild bundle, csrf-sync, express-rate-limit	REST API server — all business logic: analysis pipeline, Shopify OAuth, fix application, background scheduler, GEO scan, webhook handler
lib/db	PostgreSQL, Drizzle ORM, drizzle-zod, node-postgres pool, Neon serverless driver	Schema definitions, typed query helpers, migration runner — single source of truth for all table shapes
lib/api-client-react	React Query v5 hooks, axios, CSRF token management	Type-safe frontend API client — auto-manages CSRF headers, session cookies, base URL, and request deduplication
lib/api-zod	Zod v4 schemas	Shared request/response validation — used by both server (input validation) and client (response parsing)

1.2 Deployment Topology

- Frontend (Vercel): static SPA build, CDN-distributed, SPA catch-all rewrite rule (`/* -> /index.html`). `VITE_API_URL` env var points to the Railway API.
- API server (Railway): containerised via Dockerfile, persistent Node.js 22 process. Persistent process is required — the background scheduler (`node-cron`) and in-process analysis queue are stateful.
- Database (Neon): serverless Postgres, `ap-southeast-1` region. SSL enforced. Connection via `node-postgres` pool; session store uses a separate pool connection to avoid blocking the main pool.
- Sessions: stored in Postgres via `connect-pg-simple` (table: `session`). Auto-created in dev; manual migration required in prod. Survives Railway deploys.
- Email (Resend): transactional email service for scheduled analysis reports and AI Visibility Pulse emails. `RESEND_API_KEY` required.

1.3 Data Flow Paths

Ingest: Browser -> Google OAuth -> Express session created -> Shopify OAuth install URL generated -> Shopify permission grant -> access token returned -> token AES-256-GCM encrypted -> stored in stores table -> Shopify GraphQL API called -> all products upserted into products table (raw data, no scoring).

Analysis: POST /analyze -> idempotency key checked -> job record created (status: running) -> AnalysisQueue.enqueue() -> pipeline executes for each product: rule engine (< 5ms) -> Gemini AI scoring (200-800ms) -> gap upsert -> fix generation. Store-level passes follow: consistency analysis, perception simulator, prioritised action plan. Score snapshot written. Job status -> completed.

Fix apply: POST /fixes/:id/apply -> Shopify Admin REST API PATCH -> read-back verification GET -> fix.shopifySynced updated -> product score incremented (only if Shopify write verified) -> gap marked isFixed=true via sourceGapId -> activity log entry written -> store_summaries counters refreshed.

Scheduled: node-cron fires every minute -> DB query for schedules where enabled=true AND paused=false AND nextRunAt <= now -> nextRunAt immediately advanced (prevents duplicate runs) -> executeAnalysisPipeline() -> score snapshot written -> email decision logic evaluated -> Resend API called if conditions met.

2. Database Schema

2.1 Schema Overview — All 16 Tables

All tables use text primary keys (generated by the internal generateId() function — prefixed nanoid). All store-scoped tables reference stores(id) with ON DELETE CASCADE, so deleting a store removes all associated data automatically. All timestamps include timezone (timestamptz).

Table	Primary Key	Purpose
users	id (text)	Authenticated users — Google OAuth profile, email, display name
stores	id (text)	Connected Shopify stores — domain, AES-encrypted access token, desired positioning, status
products	id (text)	Product catalog — all Shopify product fields + per-dimension scores, tags, image URL, scoring source, AI Q&A cache
gaps	id (text, deterministic)	Detected content gaps — category, severity, evidence string, impactScore, isFixed flag, ruleId

Table	Primary Key	Purpose
fixes	id (text)	Pre-generated AI improvements — original/improved content, editedContent, sourceGapId, Shopify sync state
jobs	id (text)	Analysis job tracking — status, startedAt, completedAt, idempotencyKey
activity	id (text)	Append-only event log — analysis started/completed, fix applied, product updated via webhook
store_summaries	storeId (text, FK)	Aggregated store-level scores and counts — denormalised for fast dashboard load
consistency_reports	storeId (text, FK)	Cross-product consistency analysis results — tone, category spread, vocabulary mismatches
perception_reports	storeId (text, FK)	AI perception simulation — agent narrative, unanswered questions, FAQ health, policy bodies, structured data gaps
visibility_checks	id (text)	Individual GEO citation check records — query, aiEngine, wasCited, citationUrl, queryType
scheduled_analyses	storeId (text, FK)	Scheduler configuration per store — frequency, dayOfWeek, hourUtc, timezone, emailRecipients, pause state
score_snapshots	id (text)	Immutable analysis run history — all 7 dimension scores, product counts, issue counts, email sent flag
product_score_history	id (text)	Per-product score change records — before/after scores, delta, analysisRunId
session	sid (text)	express-session records managed by connect-pg-simple

2.2 Core Table: products

The products table is the central entity. Every analysis run updates these columns. Scores default to 0 at ingest and are populated by the analysis pipeline.

Column	Type	Notes
id	text PK	nanoid — internal primary key
storeId	text FK	references stores(id) ON DELETE CASCADE
shopifyProductId	text	Shopify GID format: gid://shopify/Product/{id}
title	text NOT NULL	Current product title — updated by webhook and fix application
description	text	Raw HTML from Shopify body_html field

Column	Type	Notes
productType / vendor	text	Optional Shopify taxonomy fields
tags	text[]	Array of tag strings — updated by fix application
imageUrl / price	text	First variant price; first image src
clarityScore / completenessScore / trustScore / tagScore / overallScore	real	0-100 per-dimension scores — set by analysis pipeline
scoringSource	text	Enum: 'ai' 'rule' 'fallback' — AI scoring provenance
hasStructuredData	boolean	True if JSON-LD Product schema detected on product page
suggestedTags	text[]	AI-generated replacement tag set from Tag Optimizer
aiQaResults / aiQaCachedAt	jsonb / timestampz	On-demand AI Q&A test results and cache timestamp
detectedCategory	text	Enum: electronics fashion beauty sports food kids home general
analyzedAt	timestampz	Timestamp of last successful analysis — used to show staleness

2.3 Core Table: gaps

Gap IDs are deterministic: gap_{storeShort}_{scopeShort}_{ruleId}. This means re-running analysis upserts the same gap record rather than creating duplicates, preserving the isFixed state across runs.

Column	Type	Notes
id	text PK	Deterministic: gap_{storeId[-8:]}_{productId[-8:]}_{ruleId}
storeId / productId	text FK / text	Scoped to store; productId null for store-level gaps
category	text NOT NULL	clarity completeness trust tags policy consistency
severity	text NOT NULL	high medium low — drives fix prioritisation
title / description / suggestion	text	Human-readable gap details shown in the UI
evidence	text	Specific evidence string from rule engine (e.g. 'Title has 2 words')
impactScore	real	0-100 — how much this gap hurts AI discoverability
effortLevel	text	low medium high — implementation effort estimate
ruleId	text	Rule engine rule ID (e.g. TITLE_TOO_SHORT) — used in deterministic ID

Column	Type	Notes
isFixed	boolean	Set true when fix is applied — preserved across re-analyses

2.4 Core Table: fixes

Column	Type	Notes
id	text PK	nanoid
storeId / productId	text	Scoped to store and product
sourceGapId	text	References the exact gap this fix resolves — enables precise gap closure
type	text	description title tags structure schema
status	text	pending applied
originalContent / improvedContent	text	AI rewrite pair — shown side-by-side in UI
editedContent	text	Merchant's manual edits — used in place of improvedContent when present
explanation / roiRationale	text	Why this fix helps AI discoverability; business impact rationale
estimatedScoreImprovement	real	0-100 — score delta if fix is applied (shown as ROI signal)
shopifySynced / shopifyError	boolean / text	Whether Shopify write verified; error message if failed
appliedAt	timestamptz	Timestamp when fix was applied — null while pending

2.5 Database Indexes

- products: (storeId) — dashboard product list; (storeId, analyzedAt) — staleness queries
- gaps: (storeId) — gap list; (storeId, productId) — per-product gap queries
- fixes: (storeId) — fix list; (storeId, productId, type, status) — dedup checks during pipeline
- score_snapshots: (storeId, createdAt) — paginated history, trend chart queries
- scheduled_analyses: (enabled, paused, nextRunAt) — critical for scheduler cron tick — scans only due rows
- visibility_checks: (storeId, checkedAt) — citation trend queries
- jobs: (storeId); (storeId, idempotencyKey); (status, startedAt) — stuck job recovery on startup

3. API Endpoint Reference

All routes are prefixed with /api. Authentication is required for all routes except those listed as Public. CSRF token required on all state-mutating requests (POST/PUT/PATCH/DELETE) except webhook endpoints.

3.1 Auth & Identity

Method	Path	Auth	Description
GET	/api/csrf-token	Public	Returns CSRF token — must be fetched before any mutating request
GET	/api/auth/me	Required	Returns current session user (id, email, name, googleId)
GET	/api/auth/google	Public	Initiates Google OAuth flow — redirects to Google consent screen
GET	/api/auth/google/callback	Public	Handles Google OAuth callback — creates/updates user, sets session cookie
POST	/api/auth/logout	Required	Destroys session and clears cookie

3.2 Shopify OAuth & Webhooks

Method	Path	Auth	Description
GET	/api/shopify/install	Required	Generates Shopify OAuth install URL for the given shop domain
GET	/api/shopify/callback	Public	Handles Shopify OAuth callback — encrypts and stores access token, fetches products
POST	/api/shopify/webhooks/products-update	Public (HMAC)	Shopify PRODUCTS_UPDATE webhook — updates product fields, busts AI Q&A cache
POST	/api/shopify/webhooks/products-delete	Public (HMAC)	Shopify PRODUCTS_DELETE webhook — removes product record and cascade-deletes gaps/fixes

3.3 Stores & Products

Method	Path	Auth	Description
GET	/api/stores	Required	Returns all stores owned by the current user
GET	/api/stores/:storeId	Required	Returns store details including status, scores, productCount

Method	Path	Auth	Description
PATCH	/api/stores/:storeId	Required	Updates store settings (desiredPositioning)
DELETE	/api/stores/:storeId	Required	Disconnects store and cascade-deletes all store data
POST	/api/stores/:storeId/refresh-products	Required	Re-fetches all products from Shopify — updates existing, adds new
GET	/api/stores/:storeId/products	Required	Returns paginated product list with scores and gap counts
GET	/api/stores/:storeId/products/:productId	Required	Returns single product with all scores, gaps, and fixes
GET	/api/stores/:storeId/summary	Required	Returns store_summaries row — overall score, gap counts, benchmark comparison
GET	/api/stores/:storeId/activity	Required	Returns paginated activity feed for the store

3.4 Analysis

Method	Path	Auth	Description
POST	/api/stores/:storeId/analyze	Required	Triggers AI readiness analysis — rate limited (5/hour/user), supports Idempotency-Key header
GET	/api/stores/:storeId/jobs/:jobId	Required	Returns job status (queued / running / completed / failed) for polling
GET	/api/stores/:storeId/gaps	Required	Returns all gaps for the store, filterable by severity and category
GET	/api/ai-fallback-stats	Required	Returns AI provider usage counters since server start (gemini/openrouter/groq/failed)

3.5 Fixes

Method	Path	Auth	Description
GET	/api/stores/:storeId/fixes	Required	Returns all fixes for the store with product title, status, Shopify sync state
POST	/api/stores/:storeId/fixes/:fixId/apply	Required	Applies a single fix: writes to Shopify, verifies, updates score. Body: { improvedContent? }
POST	/api/stores/:storeId/fixes/bulk-apply	Required	Applies all pending fixes in sequence. Body: { fixIds: string[] }

3.6 Schedule, Snapshots & CSV Export

Method	Path	Auth	Description
GET	/api/stores/:storeId/schedule	Required	Returns current schedule config (or default if none set)
PUT	/api/stores/:storeId/schedule	Required	Upserts full schedule config — recomputes nextRunAt
PATCH	/api/stores/:storeId/schedule/toggle	Required	Enable/disable schedule without touching other settings
PATCH	/api/stores/:storeId/schedule/pause	Required	Soft pause/resume — preserves config, sets nextRunAt to null
DELETE	/api/stores/:storeId/schedule	Required	Removes schedule config entirely
POST	/api/stores/:storeId/schedule/test-email	Required	Sends test report email immediately to configured recipients
GET	/api/stores/:storeId/snapshots	Required	Paginated score snapshot history (limit/offset query params, max 100)
GET	/api/stores/:storeId/snapshots/:snapshotId	Required	Returns single snapshot by ID
GET	/api/stores/:storeId/snapshots/export.csv	Required	Downloads CSV of up to 500 snapshots (Google Sheets compatible)

3.7 Insights, GEO & Store Intelligence Features

Method	Path	Auth	Description
GET	/api/stores/:storeId/tag-optimizer	Required	Returns products needing tag attention with suggested replacement tag sets
GET	/api/stores/:storeId/perception	Required	Returns AI perception report — agent narrative, unanswered questions, FAQ health, structured data gaps
GET	/api/stores/:storeId/listing-readiness	Required	Runs 12 readiness checks, returns A-F grade and per-check pass/fail with recommendations
GET	/api/stores/:storeId/query-simulation	Required	AI Query Simulation — 5 buyer-intent queries with intent classification and relevance scores. 24h TTL cache
GET	/api/stores/:storeId/topical-authority	Required	Analyzes topical coverage gaps across catalog. 24h TTL + catalog-hash invalidation
GET	/api/stores/:storeId/internal-links	Required	Internal link recommendation engine — suggests cross-linking opportunities. 24h TTL

Method	Path	Auth	Description
GET	/api/stores/:storeId/lms-txt	Required	Generates LLMs.txt store identity file content for AI crawlers
POST	/api/stores/:storeId/lms-txt/deploy	Required	Deploys LLMs.txt to Shopify as an Online Store page
GET	/api/stores/:storeId/faq-schema	Required	Generates FAQ JSON-LD schema grounded in real policy page content
GET	/api/stores/:storeId/products/:productId/ai-qa	Required	Runs AI Q&A test on a product — 5 buyer questions answered by AI. Results cached per product
GET	/api/stores/:storeId/visibility-checks	Required	Returns all manual and auto GEO citation check records
POST	/api/stores/:storeId/visibility-checks	Required	Logs a manual citation check result
GET	/api/stores/:storeId/visibility-summary	Required	Returns citation rate, byEngine breakdown, lastGeoScore
POST	/api/stores/:storeId/geo/scan	Required	Triggers real 5-platform GEO scan (Tavily + SerpAPI + Bedrock + OpenAI + Gemini). Not available on demo store.
GET	/api/stores/:storeId/competitors	Required	Returns competitor analysis with positioning comparison
GET	/api/healthz	Public	Health check — returns status, uptime, DB connectivity

4. Security Architecture

4.1 Google OAuth (User Authentication)

Users authenticate via Google OAuth 2.0 (Passport.js strategy). On first sign-in, a users record is created with the Google profile ID, email, and display name. On subsequent sign-ins, the existing record is returned by googleId lookup. The session cookie (name: sid) is set with httpOnly=true, secure=true (production), sameSite=none (required for cross-origen Vercel-Railway setup), and a 7-day maxAge.

4.2 Shopify OAuth (Store Connection)

- The merchant submits their store domain (e.g. mystore.myshopify.com).
- Domain is normalised: strips https://, www., trailing slashes, and path segments. Validated against .myshopify.com pattern. Returns 400 for invalid domains.
- OAuth install URL is generated with scopes: read_products, write_products, read_content, write_content.
- After merchant approves, Shopify sends access token to /api/shopify/callback.
- Token is immediately AES-256-GCM encrypted and written to stores.accessToken — plaintext is never written to the database.
- Callback validates the OAuth state parameter to prevent CSRF on the OAuth flow.

4.3 Token Encryption (AES-256-GCM)

Shopify Admin access tokens are permanent store credentials. Storing them in plaintext would mean a database dump exposes live admin access to every connected store.

Property	Implementation	Standard
Algorithm	AES-256-GCM (authenticated encryption)	NIST SP 800-38D
Key	32-byte (256-bit) key from TOKEN_ENCRYPTION_KEY env var (hex-encoded)	64 hex chars = 32 bytes
IV	12-byte (96-bit) random IV per encryption — generated via crypto.randomBytes(12)	GCM recommended IV size
Format	base64url(iv ciphertext authTag) — stored as single text value	Compact; base64url safe for DB storage
Auth tag	16-byte GCM authentication tag — prevents ciphertext tampering	GCM integrity guarantee

Property	Implementation	Standard
Legacy support	resolveAccessToken() detects encrypted vs plaintext by buffer length — transparent migration	Zero-downtime rollout
Startup guard	Server throws on startup if TOKEN_ENCRYPTION_KEY is absent in production	Prevents silent insecurity

4.4 Session Management

- Session store: Postgres via connect-pg-simple — survives Railway container restarts and redeploys (in-memory sessions would log out users on every deploy).
- Cookie config: httpOnly=true (no JS access), secure=true (HTTPS only in prod), sameSite=none (cross-origin Vercel/Railway), maxAge=7 days.
- SESSION_SECRET: must be set in production — server throws on startup if using default dev secret, preventing accidental insecure deployments.

4.5 CSRF Protection

csrf-sync library implements the synchronised token pattern. The client fetches a token from GET /api/csrf-token and includes it as the X-CSRF-Token header on all state-mutating requests. Shopify webhook endpoints are explicitly excluded from CSRF validation (webhooks come from Shopify servers, not browsers). The CSRF middleware runs globally before all routes.

4.6 Webhook HMAC Verification

Every incoming Shopify webhook is verified before processing. The raw request body is preserved via express.json() verify callback and stored on req.rawBody. HMAC-SHA256 is computed using SHOPIFY_API_SECRET against the raw body. The computed hash is compared to the X-Shopify-Hmac-SHA256 header using crypto.timingSafeEqual() to prevent timing attacks. Invalid signatures return 401 immediately. Duplicate webhooks within 60 seconds are dropped via an in-memory dedup Map keyed on storeId:shopifyProductId:type.

4.7 HTTP Security Headers

- X-Content-Type-Options: nosniff — prevents MIME-type sniffing
- X-Frame-Options: DENY — prevents clickjacking
- X-XSS-Protection: 1; mode=block — legacy XSS filter
- Referrer-Policy: strict-origin-when-cross-origin — limits referrer leakage

- Strict-Transport-Security: max-age=63072000; includeSubDomains; preload — production only, enforces HTTPS for 2 years

4.8 Rate Limiting

- Analysis endpoint: 5 requests per hour per user (keyed on session userId, falls back to IP). Returns HTTP 429 with JSON error when exceeded. Standard rate-limit response headers included.
- Idempotency keys: analysis requests can include an Idempotency-Key header — if a job already exists for that key + storeId, the existing job is returned without triggering a new analysis. Safe for client-side retries.
- CORS: only FRONTEND_URL and localhost origins allowed. Credentials: true required for cookie-based auth cross-origin.

5. Analysis Pipeline — Technical Detail

5.1 Queue Architecture

The AnalysisQueue class (lib/analysis-queue.ts) is a simple in-process FIFO queue with a configurable concurrency limit (set to 1). Each enqueued job is a closure that runs the full pipeline for one store. setImmediate() yields to the event loop before starting each job, allowing Express to handle other requests while an analysis is running.

- Max concurrency: 1 — prevents resource contention when multiple users trigger analysis simultaneously.
- Queue depth: unbounded (waiting jobs accumulate). Second trigger while a job is running does NOT return 409 — it queues behind the running job.
- Crash recovery: on server startup, any jobs table row with status='running' is set to status='failed' and the store status is reset, preventing permanently stuck states.

5.2 Rule Engine — 6 Categories, ~40 Rules

lib/rule-engine.ts implements the deterministic scoring layer. Rules fire against each product's raw fields. Each rule produces a RuleViolation with: ruleId, category, severity, title, description, suggestion, evidence string, impactScore (0-100), and effortLevel. Score deductions are category-specific.

Category	Example Rules	Score Impact
clarity (title)	TITLE_TOO_SHORT (<3 words: -55pts), TITLE_SHORT (<6 words: -25pts), TITLE_TOO_LONG (>15 words: -10pts), TITLE_ALL_CAPS (-8pts), MODEL_NUMBER_ONLY (-40pts)	Deductions compound; minimum score 0
completeness (description)	DESC_MISSING (-80pts), DESC_TOO_SHORT (<50 words: -40pts), DESC_WEAK (<100 words: -25pts), DESC_NO_SPECS (-20pts), DESC_NO_USE_CASE (-15pts), DESC_NO_MATERIALS (-15pts), DESC_NOT_ENGLISH (-30pts)	High-severity rules heavily penalise missing or thin content
tags	TAGS_MISSING (0 tags: -70pts), TAGS_TOO_FEW (<3 tags: -40pts), TAGS_NOT_ENOUGH (<5 tags: -20pts), TAGS_ALL_GENERIC (-30pts), TAGS_NO_CATEGORY (-25pts)	Tag quality affects AI taxonomy classification directly
trust	NO_STRUCTURED_DATA (-25pts), NO_REVIEWS (-15pts), NO_VENDOR (-10pts), NO_PRICE (-20pts), NO_PRODUCT_TYPE (-15pts)	Trust signals affect AI confidence in recommending the product
policy	NO_POLICY_FOUND — detected via perception report analysis of store pages	Policy presence affects store-level trust score

Category	Example Rules	Score Impact
consistency	Detected by store-level consistency analysis pass — cross-product tone, vocabulary, and category spread	Store-level category score, not per-product

5.3 AI Scoring Layer — 60/40 Blend

After the rule engine, each product is sent to Gemini 2.0 Flash with a structured prompt requesting JSON: { clarityScore, completenessScore, trustScore, tagScore, overallScore, summary, detectedCategory }. The prompt includes product title, description, tags, vendor, type — but NOT the rule engine's scores, to prevent circular reasoning.

- Final score formula: $\text{score} = (0.6 * \text{ruleScore}) + (0.4 * \text{aiScore})$ for each dimension. Overall = weighted average of all dimensions.
- scoringSource column records whether the final score used AI ('ai'), rule-engine fallback ('rule'), or hard fallback ('fallback').
- Scoring concurrency: CONCURRENCY = 5 — up to 5 products scored in parallel per analysis run. Balances throughput against Gemini rate limits.
- Timing: rule engine < 5ms per product; AI scoring 200-800ms per product. 50-product store: 60-90 seconds total. 500-product store: 5-8 minutes.

5.4 Gap Detection — Deterministic IDs

Gaps are upserted (not replaced) using deterministic IDs: `gap_{storeId[-8:]}_{productId[-8:]}_{ruleId}`. This design means that running the same analysis twice does not create duplicate gaps, and the isFixed flag set by a fix application is preserved across re-analyses. At the end of each analysis, stale gaps (IDs not in the current run's upserted set) are soft-deleted.

5.5 Fix Generation

For each high-severity gap, Gemini generates an improved version of the affected field. The prompt specifies: the original content, the specific rule violation, the target AI agent persona, and an instruction to preserve brand voice and factual claims. Fixes are stored pre-generated — when the merchant clicks Apply, there is zero additional AI latency.

- sourceGapId is stored on each fix, enabling precise gap closure: applying a fix marks that exact gap as isFixed=true without affecting sibling gaps in the same category.

- editedContent column: if the merchant edits the fix before applying, the edited version is used as the Shopify payload and stored separately from improvedContent.
- estimatedScoreImprovement: set per fix during generation — drives the 'sort by ROI' order in the Fixes UI.

5.6 Store-Level Analysis Passes

- Consistency analysis: cross-product tone, vocabulary consistency, category spread. Written to consistency_reports table.
- Perception simulation: AI simulates how a shopping agent perceives the store — produces narrative, ambiguities, unanswered questions, perceived strengths. Written to perception_reports table.
- Policy analysis: fetches store policy pages (refund, shipping, privacy, FAQ) and evaluates quality. Policy bodies stored in perception_reports.policyBodies for reuse by listing readiness.
- Prioritised action plan: buildPrioritizedActionPlan() ranks all gaps by (impactScore * urgency weight), produces top 5 actionable recommendations. Written to store_summaries.prioritizedActionPlan.
- Score snapshot: appended to score_snapshots table — immutable history record used for trend charts and CSV export.

6. AI Provider Architecture

6.1 Provider Cascade (lib/ai-client.ts)

chatCompletion() tries providers in order until one returns a non-empty response. Latency is measured per call. Provider usage is tracked in an in-process counter object (_fallbackStats) exposed via the /api/ai-fallback-stats endpoint for operational monitoring.

Provider	Model	Role	Env Var Required
Google Gemini	gemini-2.0-flash	Primary — analysis scoring, fix generation, perception simulation, store intelligence features	GEMINI_API_KEY
OpenRouter	google/gemma-4-31b-it:free	Fallback 1 — activates when Gemini is unavailable or returns empty	OPENROUTER_API_KEY
Groq	llama-3.3-70b-versatile	Fallback 2 — final fallback before full AI failure path	GROQ_API_KEY
OpenAI (direct)	gpt-4o-mini	Independent GEO scan judge — content quality	OPENAI_API_KEY

Provider	Model	Role	Env Var Required
		assessment. NOT part of the cascade.	
AWS Bedrock	amazon.nova-lite-v1:0	Independent GEO scan judge — brand knowledge and purchase-intent. NOT part of the cascade.	AWS credentials

6.2 Gemini Vision (Image Quality Analysis)

Product images are analysed using Gemini's multimodal capability. The image URL is fetched and sent alongside a prompt asking for: image quality score (0-100), identified issues (poor lighting, cluttered background, low resolution, no product isolation), and improvement suggestions. If Gemini API is unavailable, imageQualityScore defaults to 70 ('image present, quality unknown') rather than penalising the product.

6.3 Catalog-Hash Cache Invalidation ([lib/catalog-hash.ts](#))

Store Intelligence features (query simulation, topical authority, internal link recommendations) involve expensive AI calls that take 10-30 seconds. These results are cached in Postgres with a 24-hour TTL. However, a 24-hour TTL alone would serve stale results after a merchant applies fixes. The catalog-hash mechanism solves this:

- On each feature request, a SHA-256 hash of all product (id, title, description[:200], productType, tags) is computed and compared to the cached hash.
- If the hash differs (products added, removed, or content changed), the cache is invalidated regardless of TTL — the feature runs fresh.
- If the hash matches, the cached result is returned with cached=true and the cache timestamp, so the UI can display 'Showing cached result from [timestamp]'.
- A per-store generation lock (Map<string, Promise>) prevents concurrent requests from both triggering a fresh generation simultaneously (double-write race prevention).

6.4 AI Schema Resilience

All AI calls use Zod v4 schemas to parse and validate the JSON response. If the AI returns malformed JSON, partial output, or a response that fails schema validation, the error is caught and the fallback chain is triggered. The test suite (src/__tests__/ai-schema-resilience.test.ts) specifically exercises truncated, empty, and malformed AI responses to verify graceful degradation.

7. Background Scheduler

7.1 Architecture (lib/scheduler.ts)

The scheduler uses node-cron with a `'* * * * *'` expression (every minute). On each tick, it queries the `scheduled_analyses` table for rows where `enabled=true` AND `paused=false` AND `nextRunAt <= now`. For each due schedule, `nextRunAt` is immediately advanced before the analysis starts — this prevents duplicate runs if the analysis takes longer than 1 minute.

7.2 computeNextRun Algorithm

`computeNextRun()` computes the next UTC timestamp using `date-fns-tz` for timezone-correct arithmetic. The `hourUtc` column stores the desired run hour in the store's local timezone (despite the column name — the UI captures local time).

- Daily: sets target hour, advances by 1 day if the candidate time is already past.
- Weekly: finds the next occurrence of `schedule.dayOfWeek` (0=Sun, 6=Sat) at `schedule.hourUtc`. If today is the target day but the hour has passed, advances 7 days.
- Monthly: sets `schedule.dayOfMonth` (capped at 28 to handle short months). If the candidate date is past, advances to the same day next month.
- All results are converted back to UTC via `fromZonedTime()` for storage in `nextRunAt`.

7.3 Email Decision Logic

After each scheduled analysis completes, the scheduler evaluates whether to send an email:

- No recipients configured: snapshot is updated with `emailSkippedReason='no_recipients'`. No email sent.
- `emailOnlyOnChange=true` and `|currentScore - previousScore| < minDeltaToNotify`: skipped with reason='no_change'.
- `emailOnlyOnRegression=true` and `delta >= 0` (no score drop): skipped with reason='regression_only'.
- If all checks pass: `sendScheduledReport()` is called via Resend API. Score deltas (e.g. 'Overall: 64 to 71') are included if a previousScores snapshot exists.
- After the main report: `sendVisibilityPulse()` sends a supplementary AI-focused email with `topIssues` and `topRecommendations` from the store's prioritisedActionPlan. Failure is non-fatal (logged, does not retry).

7.4 Crash Recovery

On server startup (lib/index.ts), the jobs table is queried for any row with status='running'. These represent analyses that were interrupted by a server crash or deploy. Each stuck job is set to status='failed', the associated store's status is reset to 'connected', and an activity log entry is written. The UI detects the failed state and shows a re-trigger prompt.

8. Failure Handling & Graceful Degradation

8.1 AI Provider Failures

- Gemini returns empty or malformed JSON: Zod validation fails -> retry once with same provider -> fall back to OpenRouter Gemma 4 31B -> fall back to Groq LLaMA 3.3 70B.
- All three providers fail: rule-engine-only score returned (scoringSource='fallback'). Gap cards still shown. Fixes not generated for this product. Merchant sees a usable (if partial) score.
- Image quality call fails: imageQualityScore defaults to 70 — 'image present, quality unknown'. Product is not penalised for an infrastructure gap.

8.2 Shopify API Failures

- Product ingest fails: last known product data preserved in DB. Guard prevents concurrent ingest during analysis.
- Fix write fails: shopifyError field populated on the fix record. Score NOT updated (to avoid showing improvement that didn't land). UI shows 'Sync failed — check Shopify manually'.
- Fix verification fails (write succeeded but read-back differs): shopifySynced=false, shopifyError='Update written but Shopify verification failed'. Merchant alerted.
- Webhook HMAC invalid: 401 returned immediately. Body not parsed. No state change.

8.3 GEO Scanner Failures

- Individual agent (Tavily, SerpAPI, Bedrock, OpenAI, Gemini) fails: that agent's result is omitted. Citation rate is calculated from remaining agents. Warning note shown in UI.
- All agents fail: error returned to UI. No visibility check records written.
- Paid API quota exhausted: same as agent failure — graceful omission rather than blocking the feature.

8.4 Infrastructure Failures

Failure Mode	System Behaviour	User Experience
Gemini API down	Falls back to OpenRouter, then Groq	No visible error; slightly lower fix quality

Failure Mode	System Behaviour	User Experience
All AI providers down	Rule-engine-only score; gaps shown; fixes not generated	Score flagged as partial; fixes unavailable
Shopify API write fails	shopifyError logged; score NOT updated	Banner: 'Sync failed — check Shopify manually'
GEO scan agent fails	That agent omitted; others continue	Citation rate from available agents; warning note
Server restart mid-analysis	Job marked failed on startup; store status reset	Stale score with warning; re-trigger prompt shown
Session store (Postgres) down	Sessions lost if Postgres unreachable	Merchants logged out; must re-auth after DB recovery
Scheduled analysis fails	Error snapshot written; emailSkippedReason set; activity log entry	No email sent for failed run; merchant can re-run manually

9. Environment Variables

The following environment variables must be configured on the API server (Railway). Frontend variables (prefixed VITE_) are configured in Vercel.

Variable	Required	Description
DATABASE_URL	Required	Postgres connection string (Neon) — includes SSL params. Used by both main pool and session pool.
SESSION_SECRET	Required (prod)	Random secret for express-session cookie signing. Server throws on startup if absent in production.
TOKEN_ENCRYPTION_KEY	Required (prod)	64-char hex string (32 bytes) for AES-256-GCM Shopify token encryption. Generate: <code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code>
GOOGLE_CLIENT_ID	Required	Google OAuth 2.0 client ID from Google Cloud Console
GOOGLE_CLIENT_SECRET	Required	Google OAuth 2.0 client secret
GOOGLE_CALLBACK_URL	Required	Full URL to /api/auth/google/callback (Railway production URL)
SHOPIFY_API_KEY	Required	Shopify Partners app API key — used in OAuth install URL construction
SHOPIFY_API_SECRET	Required	Shopify Partners app secret — used for webhook HMAC verification and OAuth flow

Variable	Required	Description
GEMINI_API_KEY	Required	Google AI Studio API key — primary AI provider for scoring and fix generation
OPENROUTER_API_KEY	Recommended	OpenRouter API key — Fallback 1 for AI scoring (Gemma 4 31B free tier)
GROQ_API_KEY	Recommended	Groq API key — Fallback 2 for AI scoring (LLaMA 3.3 70B)
OPENAI_API_KEY	Required for GEO	OpenAI API key — GPT-4o-mini used as independent GEO scan content judge
TAVILY_API_KEY	Required for GEO	Tavily Search API key — live web citation checking in GEO scanner
SERP_API_KEY	Required for GEO	SerpAPI key — Google organic results in GEO scanner
AWS_ACCESS_KEY_ID	Required for GEO	AWS credentials for Bedrock Nova Lite (brand knowledge judge in GEO scanner)
AWS_SECRET_ACCESS_KEY	Required for GEO	AWS secret for Bedrock access
AWS_REGION	Required for GEO	AWS region where Bedrock Nova Lite is available (e.g. us-east-1)
RESEND_API_KEY	Required for email	Resend transactional email API key — scheduled analysis reports and visibility pulse
FRONTEND_URL	Required	URL of the Vercel SPA deployment — used for CORS origin whitelist and OpenRouter HTTP-Referer header
APP_URL	Optional	Base URL for dashboard deep-links in emails (defaults to https://app.kasparro.com)
NODE_ENV	Set by Railway	production development — controls secure cookies, session table creation, encryption key enforcement
VITE_API_URL	Vercel only	API server Railway URL — injected into SPA build for API calls

10. Key Implementation Decisions

Why Express, Not Next.js or a Serverless Architecture

Three hard requirements ruled out serverless: (1) the analysis pipeline runs 60-480 seconds per store — far beyond any serverless function timeout; (2) node-cron requires a persistent process — serverless functions are stateless and ephemeral; (3) the AnalysisQueue holds state between requests. Railway provides a persistent Node.js container. The React SPA is deployed separately on Vercel (where serverless is fine for static asset serving), and communicates with the API via CORS-enabled REST calls.

Why Drizzle ORM, Not Prisma

Drizzle generates TypeScript types directly from the schema file without a separate code-generation step. Queries compile to readable SQL with full TypeScript inference. Prisma's migration system enforced opinions that conflicted with IF NOT EXISTS semantics needed for in-place table additions on an existing Neon database. Drizzle's schema-as-code approach also makes it straightforward to add columns and review the generated SQL before applying.

Why Postgres Sessions, Not JWT or Redis

Railway containers restart on deploy. In-memory sessions (the express-session default) would log out every merchant on every deployment. JWTs solve the statefulness problem but require a token invalidation mechanism (logout, revocation list) which re-introduces state. Postgres sessions with connect-pg-simple are persistent, survive restarts, and are invalidatable via direct DB deletion. Redis would be better at scale (lower latency) but adds a deployment dependency not justified at this stage.

Why Pre-Generated Fixes, Not On-Demand

Generating a Gemini rewrite at fix-apply time introduces AI latency (200-800ms) at the worst possible UX moment — when the merchant has just decided to act. Pre-generating fixes during the analysis pipeline means Apply is instant: the fix payload is already in the DB. The tradeoff is that fixes generated during analysis reflect the product state at analysis time; if the merchant edits the product after analysis but before applying, the fix may be slightly stale. This is documented and the editedContent field handles manual correction.

Why Deterministic Gap IDs

The naive approach — delete all gaps and re-insert on each analysis run — would lose the isFixed state set when a merchant applies a fix. If a gap is fixed, re-running analysis should not mark it as broken again.

Deterministic IDs (`gap_{storeId[-8:]}_{productId[-8:]}_{ruleId}`) make upsert semantics straightforward: the same rule on the same product always produces the same ID, so `ON CONFLICT DO UPDATE` preserves `isFixed` while refreshing the evidence, suggestion, and `impactScore`.

Why Idempotency Keys on Analysis

Mobile clients and unstable connections may retry requests. Without idempotency, a retry during a running analysis would queue a second analysis behind the first — wasting resources and producing confusing results (two snapshots in quick succession). With the `Idempotency-Key` header, a retry within the same key returns the existing job ID and status without creating a new job. The key is stored in the jobs table and looked up before queue insertion.

In-Process Mutex for Feature Generation

Store Intelligence features are expensive (10-30 seconds) and cached. If two browser tabs open the same feature simultaneously, without a lock both tabs would fire the AI call and race to write the cache. The in-process `Map<string, Promise<T>>` (`generationLocks`) solves this: the second caller receives the same Promise as the first, awaiting the same generation. The lock releases when the Promise settles. This is per-process — two Railway instances would still race. At single-instance scale this is sufficient; multi-instance would require a distributed lock (Redis `SETNX` or similar).

11. Known Limitations & Production Roadmap

Current Limitations

In-Process State Is Not Durable

The `AnalysisQueue`, generation locks, and webhook dedup map all live in the Node.js heap. A process crash wipes them. Crash recovery handles stuck jobs at startup, but in-flight dedup state and generation locks are lost. Production resolution: BullMQ on Redis for the analysis queue (durable, horizontally scalable), Redis `SETNX` for distributed locks, and Redis expiry for webhook dedup.

No Integration or End-to-End Tests

Unit tests cover the rule engine and AI schema resilience. There are no integration tests for the Shopify OAuth flow, the full analysis pipeline, or the fix application and read-back verification sequence. The most common production failure modes (Shopify API errors, AI provider timeouts, Neon cold starts) are not exercised in CI. This is the largest production readiness gap.

GEO Scanner Depends on External Paid APIs

Tavily and SerpAPI require API keys with daily quota limits. On free tiers, a GEO scan for a large store can exhaust the daily allowance. The scanner gracefully omits individual agents that error, but exhausted quotas are indistinguishable from API failures. Production resolution: quota tracking per-store per-day in Postgres, surfaced in the UI as 'X scans remaining today', with alerts at 80% consumption.

Score Trend Polluted by Failed Analysis Runs

Every analysis writes a score snapshot, even partial failures. A run where all AI providers fail produces rule-engine-only scores that can appear lower than the previous run, incorrectly showing a regression. We filter snapshots with overallScore=null, but partial-failure scores (AI failed but rule engine ran) appear as real data points. Production resolution: add a confidence enum column to score_snapshots (full | partial | rule_only | failed) and allow filtering by confidence in the trend chart.

60/40 Score Blend Ratio Is Intuition-Based

The rule-engine-to-AI score blend (60% rules, 40% AI) was chosen by experimentation on a small product sample. It has not been validated against real AI citation outcomes. The right question is: does a higher Kasparro score correlate with more GEO scan citations? If not, the ratio needs tuning. Production resolution: instrument GEO scan results against Kasparro scores across a large store cohort and optimise the blend weights using the correlation data.

LLMs.txt Deployment Is Manual for Most Stores

The LLMs.txt generator produces a machine-readable store identity file per the emerging AI crawler standard. Deploying it to the root URL of a Shopify store requires theme code access (modifying routes.json or adding a custom template) — outside the scopes granted by the current Shopify OAuth flow (read_products, write_products only). The feature generates the content but relies on the merchant to follow a manual deployment guide. Production resolution: request the themes scope and build a one-click theme injection via the Shopify Theme API.